

协议数据的捕获和解析实验报告

LaTeX by 王子轩 (学号: 2022211210 班级:2022211301)

2024 年 6 月 8 日

目录

1	实验内容和实验环境描述	3
1.1	实验任务和实验内容	3
1.2	实验环境	3
1.3	实验步骤概述	3
2	详细实验步骤和协议分析	3
2.1	IP 协议分析	3
2.1.1	捕获 ICMP 数据包	3
2.1.2	IP 包头校验和的校验原理	5
2.1.2.1	具体 IP 包头的校验和验证	5
2.1.3	IP 包分段原理与传输过程	6
2.2	ICMP 协议分析	9
2.2.1	ICMP 功能	9
2.2.1.1	传递错误消息	9
2.2.1.2	传递操作消息	9
2.2.1.3	支持路径 MTU 发现	9
2.2.1.4	提供网络诊断工具	10
2.2.2	ICMP 包格式	10
2.3	DHCP 协议分析	10
2.3.1	DHCP 功能	10
2.3.1.1	动态分配 IP	11
2.3.1.2	多种配置	11
2.3.1.3	IP 租期管理	11
2.3.1.4	静态分配与自动分配 IP	11
2.3.2	捕获 DHCP 报文	11
2.3.3	DHCP 协议字段和过程分析	12
2.3.4	消息序列图	15
2.3.5	进一步分析	15
2.4	ARP 协议分析	16
2.4.1	ARP 各字段功能	16
2.4.2	ARP 通信过程	16

2.5	TCP 协议分析	17
2.5.1	TCP 首部字段功能	17
2.5.2	捕获 TCP 数据包	18
2.5.3	连接建立	18
2.5.4	连接断开	21
2.5.5	数据传输过程	24
3	实验结论和心得	25

1 实验内容和实验环境描述

1.1 实验任务和实验内容

本次实验主要包含下列内容:

1. 使用 Wireshark 软件捕获在使用 ping 命令时产生的 ICMP 消息;
2. 分析网络层 IP 包头格式, 理解各字段的作用, 对于分段和校验和进行验证;
3. 使用 Wireshark 软件捕获在使用 ARP 消息, 分析其消息格式, 理解其工作原理;
4. 使用 Wireshark 捕获 DHCP 消息, 分析其消息序列, 理解 DHCP 的功能和操作原理;
5. 使用 Wireshark 捕获 TCP 消息, 分析 TCP 报文段头格式, 理解连接建立和释放的原理, 差错控制原理、序号和窗口管理的原理。

通过本实验可以深入理解分层体系结构, 理解和掌握 TCP/IP 协议栈的代表协议——IP、TCP、UDP、ICMP、ARP 和 DHCP 协议的要点。

1.2 实验环境

Windows 11

Wireshark Version 4.2.2

1.3 实验步骤概述

运行 Wireshark 软件, 在启动页中选中活跃的网卡, 如以太网, 然后设置捕获过滤器条件, 可以设置需要捕获的协议和端口号, 例如捕获 DHCP 消息时, 应设为: udp port 67。设置好后, 点击左上角蓝色的图标即开始捕获。

启动捕获之后, 运行相应的网络通信程序, Wireshark 即可以捕获到网卡发送和接收到的符合捕获条件的数据, 然后通过 wireshark 工具的 PacketList, PacketDetail, PacketByte 窗口进行解码分析, 从而完整地了解不同协议的通信过程和应用场景

2 详细实验步骤和协议分析

2.1 IP 协议分析

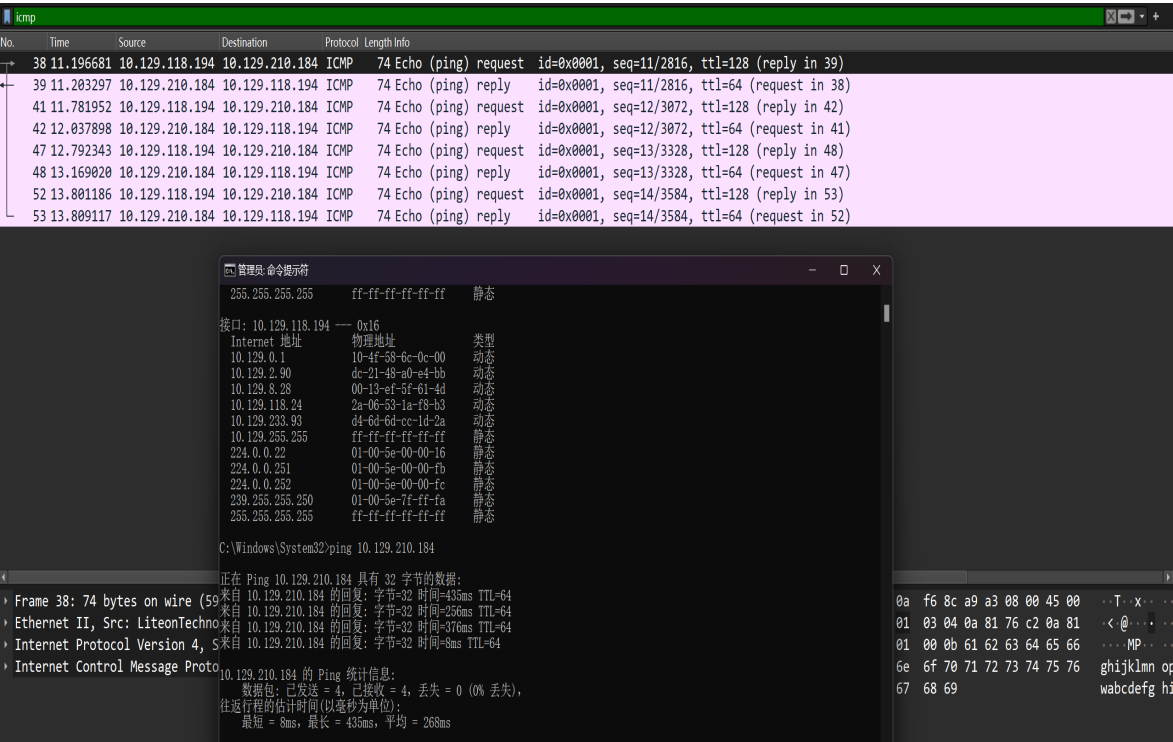
2.1.1 捕获 ICMP 数据包

首先查看自己的网关地址与子网掩码, 确认自己所在的网段范围

```
无线局域网适配器 WLAN:

    连接特定的 DNS 后缀 . . . . . : 
    IPv6 地址 . . . . . : 2001:da8:215:3c0a:6a4f:75e:90d2:a247
    临时 IPv6 地址 . . . . . : 2001:da8:215:3c0a:59c8:450:40be:1e49
    本地链接 IPv6 地址 . . . . . : fe80::a32:9a27:2dd0:fae0%22
    IPv4 地址 . . . . . : 10.129.118.194
    子网掩码 . . . . . : 255.255.0.0
    默认网关 . . . . . : fe80::104f:5883:856c:c00%22
                        10.129.0.1
```

设置 Wireshark 的显示过滤器为 icmp。对同一无线网中另一台设备如 10.129.210.184(我的平板) 执行 ping 命令, 该设备此前不在本设备的 ARP 表中。



选择第一个数据包中的 IP 头部进行分析

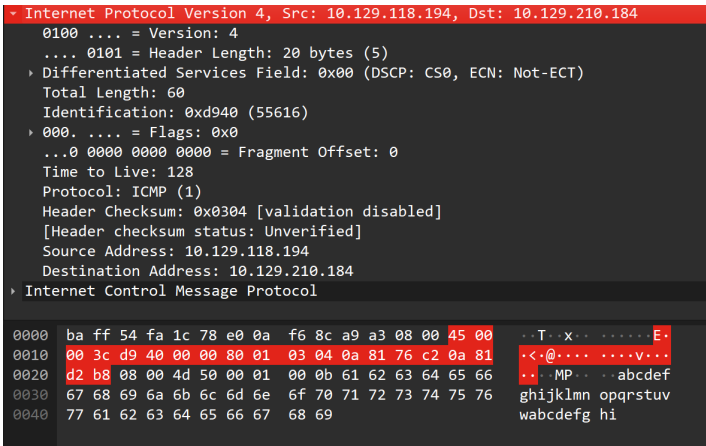


图 1: 序号为 38 的分组

其中包头各字段的含义

字段	报文 (16 进制)	内容
包头长度	45	20 字节 (5 * 32 位/8)
服务类型	00	DSCP: CS0 , ECN: Not ECT 默认优先级 不支持ECN拥塞管理机制 正常时延 正常吞吐量 正常可靠性
总长度	003c	60 字节
标识	d940	55616
标志	00	DF = 0 MF = 0 允许分片, 此片为最后一个分片
片偏移	00	位偏移为0
生存周期	80	每一跳生存时间为128s
协议	01	携带的数据来自ICMP (1) 协议
头部校验和	0304	IP头部校验和为0x0304
源地址	0a 81 76 c2	源地址为10.129.118.194
目标地址	0a 81 d2 b8	目的地址为10.129.210.184

图 2: 序号为 38 的分组 IP 包头详细信息

2.1.2 IP 包头校验和的校验原理

IP 包头校验和 (IP Header Checksum) 用于确保 IP 数据包头部的完整性。它通过对 IP 头部的每 16 位字段进行求和, 并取反码 (反转所有位) 来计算。

详细计算过程如下:

- 1. 初始化: 注意, 此时, 将 IP 头部中的校验和字段初始化为 0。这是为了确保在计算校验和时, 不包括原始的校验和值。
- 2. 将 IP 头部分成 16 位的块: 将 IP 头部的数据 (通常是 20 字节) 划分成若干个 16 位 (2 字节) 的块。如果头部的字节数是奇数, 需要在最后加一个填充字节, 使其变为偶数个字节。
- 3. 求和: 将所有的 16 位块按二进制补码进行求和。由于计算机的位宽限制, 这个求和过程可能会产生进位 (超过 16 位的部分), 需要将这些进位加回到结果中。
如果求和结果超过 16 位, 将高于 16 位的部分 (进位) 加到低 16 位。
- 4. 进位处理: 在每次求和过程中, 如果产生了进位 (溢出), 就把高 16 位的进位加到低 16 位。
- 5. 取反码: 将求和结果按位取反 (每一位都反转)。这就是校验和。
- 6. 将校验和填入头部: 将计算得到的校验和填入 IP 头部的校验和字段。

2.1.2.1 具体 IP 包头的校验和验证

0000	45 00 00 3c d9 40 00 00 80 01 03 04 0a 81 76 c2
0010	0a 81 d2 b8

将所有 16 位字段 (03 04 字段改为 00 00) 按规则相加, 得到结果为 FCFB, 最后各位取反

~FCFB = 0304

结果与正确校验和相符, 验证成功

2.1.3 IP 包分段原理与传输过程

IP 包分段是将一个较大的 IP 数据报分割成多个更小的数据报段, 以便在网络中传输。这种分割方式可以避免在网络中传输过程中因为 MTU 限制而导致的数据包过大无法传输的问题。

在 IP 包分段过程中, 一个数据报段的大小是由 MTU (最大传输单元) 决定的。如果一个 IP 数据报的大小超过了 MTU, 就需要将其分割成多个数据报段, 并在每个数据报段中添加一些额外的 IP 头部信息, 以帮助接收方重新组装这些数据报段。

MTU 最大传输单元是由不同网络类型决定的, 在标准以太网 Ethernet 中, MTU 为 1500 字节, 代表数据链路层上数据段最大为 1500 字节, 而上层的网络层是由数据链路层限制的, 对于网络层来说, 它实际的“MTU”为 1480(1500-20) 字节, 通常情况下, 我们讨论的都是链路层上的 MTU。

根据所提供的 IP 包数据, 可以看出这个包并没有被分割成多个数据报段, 因为标志字段为 00, 片偏移字段为 0000, 即不分片。此外, 通过总长度字段可以知道, 这个 IP 数据报的总长度为 60 字节, 这个长度包含了 IP 头部和 ICMP 负载。如果这个 IP 数据报的长度超过了 MTU, 那么就需要将其分割成多个数据报段。

在本例中, 该 IP 数据报并没有被分割成多个数据报段, 因为标志字段为 00, 片偏移字段为 0000, 且其总长度为 60 字节, 没有超过 MTU。

于是我们准备制作新的 8000 字节的 IP 数据分组并发送, 捕获后分析其分片传输的分组结构。

通过强制使用 IPv4 地址通过 ping 命令找到的对应学校服务器的 IP 地址为 10.3.9.161

```
>ping -4 www.bupt.edu.cn
```

正在 Ping vn46.bupt.edu.cn [10.3.9.161] 具有 32 字节的数据:

来自 10.3.9.161 的回复: 字节=32 时间=2ms TTL=59

来自 10.3.9.161 的回复: 字节=32 时间=2ms TTL=59

来自 10.3.9.161 的回复: 字节=32 时间=1ms TTL=59

来自 10.3.9.161 的回复: 字节=32 时间=2ms TTL=59

10.3.9.161 的 Ping 统计信息:

数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
往返行程的估计时间(以毫秒为单位):

最短 = 1ms, 最长 = 2ms, 平均 = 1ms

输入命令 ping -l 8000 -4 www.bupt.edu.cn, 设置 Wireshark 的显示过滤器为 ip.addr==10.3.9.161 (10.3.9.161 即为 www.bupt.edu.cn 的 ip 地址)。捕获到如下内容:

可以看到, 每个 ICMP 包都被拆分成了 6 片, 链路的 MTU 是 1500。除去 IP 头后, 前五个分片每个包含 1480 字节, 最后一个分片包含 608 字节, 一共 8008 字节, 正好是 8 字节的 ICMP 头加上 8000 字节的数据

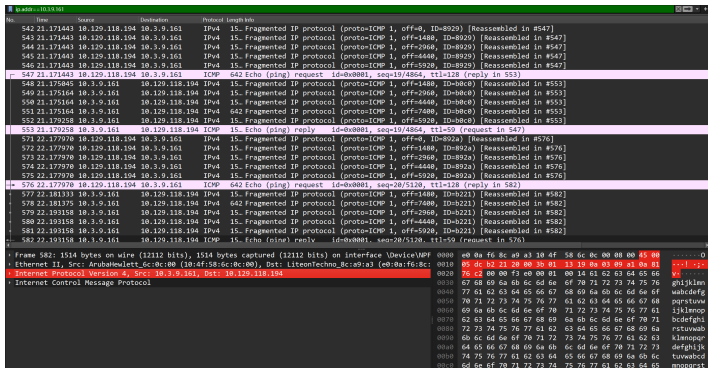


图 3: 抓包结果

以第一个包为例 (分组 542-547)
观察第一个分片的 IP 头信息 该分片中 MF 位为 1, 表示这不是一个完整的包, 后面还有更

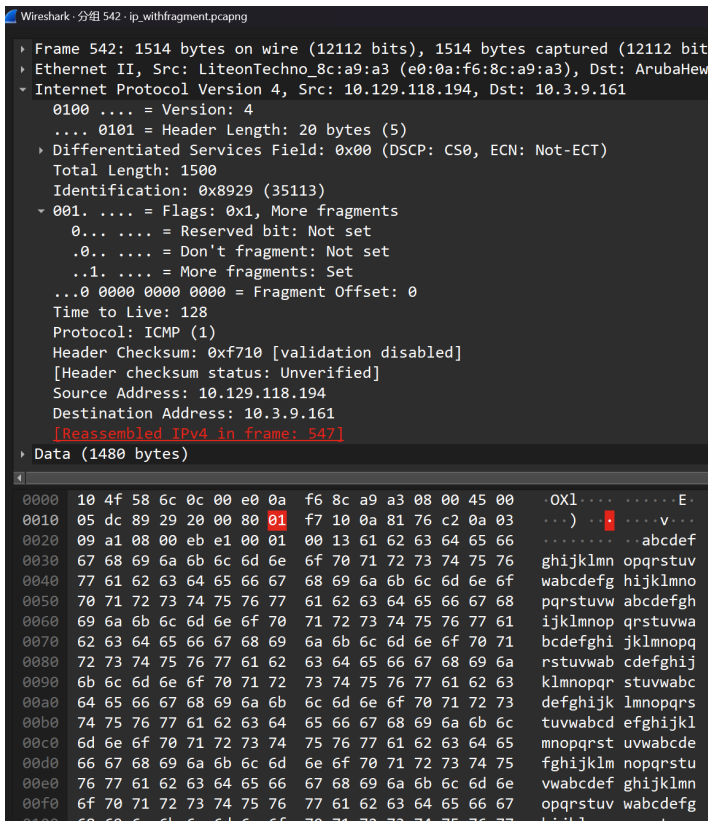


图 4: 分组 542

多分片。Fragment Offset 为 0, 表示这个分片在包中的偏移量为 0, 即第一个分片。第一个包的第二个分片与之类似, 但 Fragment Offset 变为 1480, 表示这个分片在包中的偏移量为 1480。

下面直接看第一个包的最后一个分片。这里的 MF 位变为 0，表示后面不再有更多的分片。Fragment Offset 为 7400，表示这个分片在包中的偏移量为 7400。

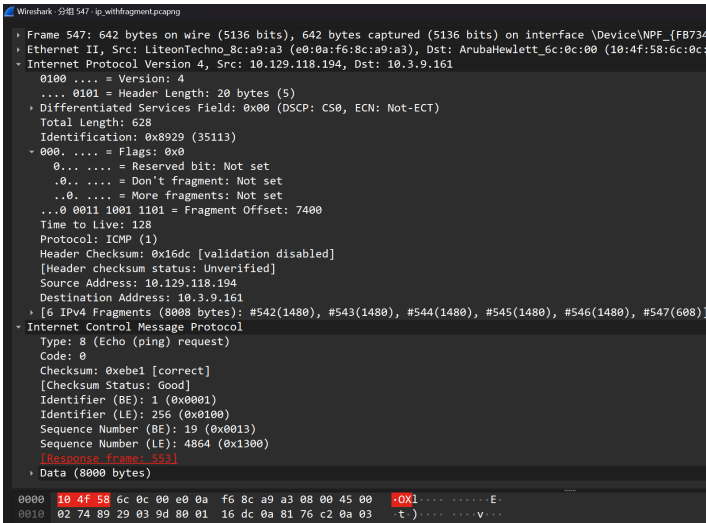


图 5: 分组 547

数据包分组的详细信息:

属性	Frame 542	Frame 543	Frame 544	Frame 545	Frame 546
帧编号	542	543	544	545	546
帧大小	1500 字节	1500 字节	1500 字节	1500 字节	1500 字节
数据部分	1480 字节	1480 字节	1480 字节	1480 字节	1480 字节
IPv4 源	10.129.118.194	10.129.118.194	10.129.118.194	10.129.118.194	10.129.118.194
IPv4 目的	10.3.9.161	10.3.9.161	10.3.9.161	10.3.9.161	10.3.9.161
片偏移	0	1480	2960	4440	5920
更多片段	是	是	是	是	是
协议	ICMP	ICMP	ICMP	ICMP	ICMP

图 6: 前五个到达的分组

字段	报文 (16 进制)	内容
包头长度	45	20 字节 (5)
服务类型	00	DSCP: CS0, ECN: Not-ECT
总长度	0274	628 字节
标识	8929	35113
标志	03	最后一个分片
片偏移	9d	偏移量为7400 (925*8)
生存周期	80	每一跳存活时间为128s
协议	01	ICMP (1)
头部校验和	16dc	[correct]
源地址	0a8176c2	10.129.118.194
目的地址	0a0309a1	10.3.9.161

图 7: 最后一个分组信息

经过重组后, 完整的 IPv4 数据包长度为 8008 字节。根据这些信息, 我们可以得知 Frame 58 与其他分段数据包共同构成了一个更大的 IP 数据包, 这些数据包在传输过程中被分段以满足网络层的最大传输单元 (MTU) 限制。在目的地, 这些分段数据包需要被重新组合以重建完整的 IP 数据包。

同一个包的所有分片都有相同的 Identification 字段, 从而可以确认哪些分片属于同一个包。

2.2 ICMP 协议分析

2.2.1 ICMP 功能

2.2.1.1 传递错误消息

ICMP 主要用于传递网络层的错误消息, 帮助网络管理员和协议实现者了解网络问题。常见的错误消息包括:

1. 目标不可达 (Destination Unreachable): 当路由器或主机无法到达目标地址时, 发送此消息。
2. 源抑制 (Source Quench): 当路由器或主机检测到网络拥塞时, 建议发送方降低发送速率。
3. 重定向 (Redirect): 当路由器发现更好的路径时, 通知发送方更新其路由表。
4. 时间超时 (Time Exceeded): 当数据包在网络中转发的时间超过了 TTL (生存时间) 值时, 发送此消息。

2.2.1.2 传递操作消息

ICMP 还用于传递操作信息, 以协助网络配置和诊断。常见的操作信息包括:

回显请求和回显应答 (Echo Request and Echo Reply): 用于 ping 命令, 测试主机间的连通性。

时间戳请求和时间戳应答 (Timestamp Request and Timestamp Reply): 用于测量两个设备之间的往返时间。

2.2.1.3 支持路径 MTU 发现

ICMP 还用于路径 MTU 发现 (Path MTU Discovery), 帮助发送方确定到达目标主机的最小 MTU, 以避免分片。

在这个过程中, 发送方会发送设置了“不分片 (Don't Fragment)”标志的数据包, 如果数据包过大, 路径上的设备会返回“需要分片但设置了不分片标志 (Fragmentation Needed and DF

Set)” 的 ICMP 消息, 通知发送方降低数据包大小。

2.2.1.4 提供网络诊断工具

ICMP 被广泛用于各种网络诊断工具, 包括:

ping: 通过发送 ICMP 回显请求和接收回显应答, 测试主机间的连通性和响应时间。

tracert: 通过发送 ICMP 数据包逐步增加 TTL 值, 跟踪数据包到达目标主机的路径, 并识别路径上的每个路由器。

2.2.2 ICMP 包格式

利用 IP 协议分析中一开始捕获到的 ICMP 数据包进行 ICMP 包格式分析

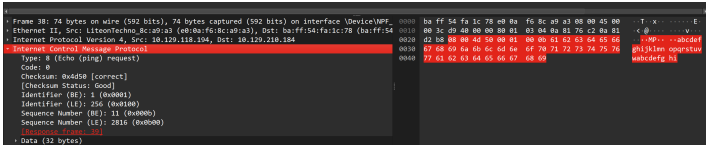


图 8: 协议格式

ICMP 包前四个字节是统一格式, 共有三个字段: 类型、代码与校验和。后续四个字节与 ICMP 的类型有关, 共有头部 8 个字节

在本例中, ping 发送包作为 DHCP ehco request 包格式为

Type:8 (Echo (ping) request) 回显请求消息
Code: 0 标准回显请求
Identifier (BE): 1 (0x0001)
Identifier (LE): 256 (0x0100) 不同字节序机器中用于匹配请求和应答的标识符
Sequence Number (BE): 11 (0x000b)
Sequence Number (LE): 2816 (0x0b00) 标识发送的顺序

在本例中, ping 接收包作为 DHCP ehco reply 包格式为

Type:0 (Echo (ping) reply) 回显相应消息
Code: 0 标准相应请求
Identifier (BE): 1 (0x0001)
Identifier (LE): 256 (0x0100) 不同字节序机器中用于匹配请求和应答的标识符
Sequence Number (BE): 11 (0x000b)
Sequence Number (LE): 2816 (0x0b00) 标识发送的顺序

另外, 注意到其是携带的数据内容是 abcdefg..., 对方回应的 ICMP 报文携带的内容也要完全相同。

2.3 DHCP 协议分析

2.3.1 DHCP 功能

DHCP (Dynamic Host Configuration Protocol, 动态主机配置协议) 是一种网络管理协议, 用于自动分配 IP 地址和其他网络配置参数给网络上的设备, 使它们能够在 IP 网络中进行通信。

2.3.1.1 动态分配 IP

DHCP 服务器可以动态分配 IP 地址给网络中的客户端设备。这个过程包括以下步骤:

- 发现 (Discover): 客户端设备发送 DHCP 发现消息, 寻找可用的 DHCP 服务器。
- 提供 (Offer): DHCP 服务器响应发现消息, 提供一个可用的 IP 地址和其他配置参数。
- 请求 (Request): 客户端设备选择一个提供的 IP 地址, 并向服务器发送请求消息以确认。
- 确认 (Acknowledge): DHCP 服务器确认请求, 并将 IP 地址和配置参数正式分配给客户端设备。

2.3.1.2 多种配置

除了 IP 地址, DHCP 还可以提供其他网络配置参数, 包括:

- 子网掩码 (Subnet Mask): 定义 IP 地址的网络部分和主机部分。
- 默认网关 (Default Gateway): 用于将流量转发到其他网络。
- DNS 服务器 (DNS Server): 用于解析域名到 IP 地址。
- NTP 服务器 (NTP Server): 用于同步时间。
- 其他配置: 如 TFTP 服务器、域名、NetBIOS 名称服务器等。

2.3.1.3 IP 租期管理

DHCP 服务器分配的 IP 地址有一个租期 (Lease), 即 IP 地址的使用时间。租期结束后, 客户端需要重新请求 IP 地址。这个机制确保了 IP 地址的有效利用, 防止 IP 地址浪费。

2.3.1.4 静态分配与自动分配 IP

除了动态分配, DHCP 服务器还可以基于 MAC 地址进行静态分配 (Static Allocation)。这意味着特定设备每次都收到相同的 IP 地址。这对于需要固定 IP 地址的设备 (如服务器、打印机等) 非常有用。

在一些场景下, DHCP 服务器还可以自动分配 IP 地址, 而无需客户端发起请求。这种方式通常用于初次配置网络设备时。

2.3.2 捕获 DHCP 报文

- 通信双方的端口为
 - UDP 端口 67: DHCP 服务器监听此端口, 接收来自 DHCP 客户端的请求。UDP 端口 68: DHCP 客户端监听此端口, 接收来自 DHCP 服务器的响应
- 将 Wireshark 显示过滤器设置为 `udp.port==68`, 先使用 `ipconfig /release` 命令释放已经申请的 IP 地址, 然后执行 `ipconfig /renew` 获取 IP 地址。
- 在 Wireshark 中抓到以下报文

No.	Time	Source	Destination	Protocol	Length	Info
498	8.003659	10.129.118.194	10.3.9.2	DHCP	342	DHCP Release - Transaction ID 0xa87faed6
29...	33.081322	0.0.0.0	255.255.255.2...	DHCP	342	DHCP Discover - Transaction ID 0xc2162de3
35...	36.774809	0.0.0.0	255.255.255.2...	DHCP	342	DHCP Discover - Transaction ID 0xc2162de3
39...	40.677524	0.0.0.0	255.255.255.2...	DHCP	342	DHCP Discover - Transaction ID 0xc2162de3
41...	42.618179	0.0.0.0	255.255.255.2...	DHCP	342	DHCP Discover - Transaction ID 0x8366b862
44...	47.485250	0.0.0.0	255.255.255.2...	DHCP	342	DHCP Discover - Transaction ID 0x8366b862
44...	47.493251	10.3.9.2	10.29.120.253	DHCP	342	DHCP Offer - Transaction ID 0x8366b862
44...	47.494187	0.0.0.0	255.255.255.2...	DHCP	354	DHCP Request - Transaction ID 0x8366b862
44...	47.515454	10.3.9.2	10.29.120.253	DHCP	342	DHCP ACK - Transaction ID 0x8366b862

- 第一个报文是在执行 `release` 命令时, 释放 IP 地址的过程
- 当前客户端通知 DHCP 服务器其放弃 (释放) 之前分配的 IP 地址
- 之后的报文是执行 `renew` 命令, 客户端申请 IP 地址的过程, 下面进行具体分析

2.3.3 DHCP 协议字段和过程分析

DHCP 报文格式为

0										1										2										3									
0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1								
操作码										硬件类型										硬件地址长度										跳数									
事务ID																																							
等待时间																																							
标志字段																																							
客户端IP地址																																							
你的IP地址																																							
下一个服务器IP地址																																							
网关IP地址																																							
客户硬件地址																																							
服务器主机名																																							
引导文件名																																							
可选参数字段																																							

首先客户端广播 DHCP Discover 报文，在本例中，客户端一开始未能及时收到 DHCP 服务器的响应时，通过多次发送 Discover 消息，客户端增加了获得 DHCP 服务器响应的机会，从而确保网络连接的可靠性，这属于正常现象

重点分析最后一个 Discover 报文

注意到 IP 包头中源地址为 0.0.0.0，这是由于客户端当前还没有分配到 IP 地址，故使用 0.0.0.0 占位；IP 包头中目的地址为 255.255.255.255，这是由于客户端不知道 DHCP 服务器的 IP 地址，必须进行广播。

重点分析该报文中的一些值得讨论的字段：1. Message type 只能为 1 或 2，1 表示客户发出的请求，2 表示服务端的应答。

```

# Frame 4431: 342 bytes on wire (2736 bits), 342 bytes captured (2736 bits) on interf
# Ethernet II, Src: LiteonTechno_8c:a9:a3 (e0:0a:f6:8c:a9:a3), Dst: Broadcast (ff:ff:
# Internet Protocol Version 4, Src: 0.0.0.0, Dst: 255.255.255.255
# User Datagram Protocol, Src Port: 68, Dst Port: 67
# Dynamic Host Configuration Protocol (Discover)
  Message type: Boot Request (1)
  Hardware type: Ethernet (0x01)
  Hardware address length: 6
  Hops: 0
  Transaction ID: 0x8366b862
  Seconds elapsed: 0
  Bootp flags: 0x0000 (Unicast)
  Client IP address: 0.0.0.0
  Your (client) IP address: 0.0.0.0
  Next server IP address: 0.0.0.0
  Relay agent IP address: 0.0.0.0
  Client MAC address: LiteonTechno_8c:a9:a3 (e0:0a:f6:8c:a9:a3)
  Client hardware address padding: 00000000000000000000
  Server host name not given
  Boot file name not given
  Magic cookie: DHCP
  Option: (53) DHCP Message Type (Discover)
  Option: (61) Client identifier
  Option: (12) Host Name
  Option: (60) Vendor class identifier
  Option: (55) Parameter Request List
  Option: (255) End
  Padding: 00000000000000000000000000000000

```

2. Transaction ID (事务 ID) 为一个随机标识符，服务器的应答必须带上相同的标识符，以区分不同的 DHCP 请求。

3. Bootp flags 字段目前只有最高位被用于指定应答报文是否广播，其他位都保留备用。在本报文中最高位为 0，表示客户端希望服务器使用单播的方式回应。1 表示广播回复，0 表示单播回复。

DHCP 报文中还附带了若干选项 (Option)

下面为选项字段的格式 (其中长度表示选项数据的字节数)

[illegible]

下面是一些值得讨论的选项：

1. (Option 53) DHCP Message Type 指示了该报文的类型为 Discover
2. (Option 12) Host Name 对应客户端主机名
3. (Option 50) Requested IP Address 指示了客户端希望获得的 IP 地址，为了尽可能维持网络的稳定，客户端会尽可能要求获得和之前一样的 IP 地址，服务器会尽可能满足客户端的要求。
4. (Option 55) Parameter Request List 指示了客户端希望获取的额外信息，包括但不限于子网掩码、默认网关、DNS 服务器等

接下来是服务器发来的 DHCP Offer 报文。Offer 报文表示服务器给客户端提供了一个可供使用的 IP 地址，但最终是否使用仍需要客户端自己决定。

下面分析在这个报文中值得讨论的字段和选项：1. Hops 表示 DHCP 报文经过中继的次数。在这里 Hops 为 1，表示经过了一次中继。与之有关的字段是 Relay agent IP address，表示 DHCP 中继的地址，在这里实际上就是校园网所分配给当前主机的网关。该网关将 DHCP 报文中继给真正的 DHCP 服务器，然后再回复给客户端。

2. Your (client) IP address 表示服务端提供的 IP 地址。
3. Option(54) DHCP Server Identifier 表示 DHCP 服务器的信息，在这里就是 DHCP 服务器的 IP 地址。这个字段将被用于区分多个服务器。
4. Option(51) IP Address Lease Time 表示租约时长，展开后可看到时长为两小时。

```

Internet Protocol Version 4, Src: 10.3.9.2, Dst: 10.29.120.253
User Datagram Protocol, Src Port: 67, Dst Port: 68
Dynamic Host Configuration Protocol (Offer)
  Message type: Boot Reply (2)
  Hardware type: Ethernet (0x01)
  Hardware address length: 6
  Hops: 1
  Transaction ID: 0x8366b862
  Seconds elapsed: 0
  Bootp flags: 0x0000 (Unicast)
  Client IP address: 0.0.0.0
  Your (client) IP address: 10.29.120.253
  Next server IP address: 0.0.0.0
  Relay agent IP address: 10.29.0.1
  Client MAC address: LiteonTechno_8c:a9:a3 (e0:0a:f6:8c:a9:a3)
  Client hardware address padding: 00000000000000000000
  Server host name not given
  Boot file name not given
  Magic cookie: DHCP
  Option: (53) DHCP Message Type (Offer)
  Option: (54) DHCP Server Identifier (10.3.9.2)
  Option: (51) IP Address Lease Time
  Option: (1) Subnet Mask (255.255.0.0)
  Option: (3) Router
  Option: (6) Domain Name Server
  Option: (255) End
  Padding: 00000000000000000000000000000000

```

5. Option(1) Subnet Mask, Option(3) Router, Option(6) Domain Name Server 提供了客户端需要的额外信息, 分别为子网掩码 (255.255.0.0)、默认网关 (10.29.0.1) 和 DNS 服务器 (10.3.9.5 和 10.3.9.4 和 10.3.9.6)。

此处也可以发现服务器向客户端的响应报文是单播传送, 符合 Boots flags 字段中所定义的要求

接下来客户端确认使用这个 IP 地址, 发送 DHCP Request 报文向服务器请求使用这个 IP 地址。

```

Internet Protocol Version 4, Src: 0.0.0.0, Dst: 255.255.255.255
User Datagram Protocol, Src Port: 68, Dst Port: 67
Dynamic Host Configuration Protocol (Request)
  Message type: Boot Request (1)
  Hardware type: Ethernet (0x01)
  Hardware address length: 6
  Hops: 0
  Transaction ID: 0x8366b862
  Seconds elapsed: 0
  Bootp flags: 0x0000 (Unicast)
  Client IP address: 0.0.0.0
  Your (client) IP address: 0.0.0.0
  Next server IP address: 0.0.0.0
  Relay agent IP address: 0.0.0.0
  Client MAC address: LiteonTechno_8c:a9:a3 (e0:0a:f6:8c:a9:a3)
  Client hardware address padding: 00000000000000000000
  Server host name not given
  Boot file name not given
  Magic cookie: DHCP
  Option: (53) DHCP Message Type (Request)
  Option: (61) Client identifier
  Option: (50) Requested IP Address (10.29.120.253)
  Option: (54) DHCP Server Identifier (10.3.9.2)
  Option: (12) Host Name
  Option: (81) Client Fully Qualified Domain Name
  Option: (60) Vendor class identifier
  Option: (55) Parameter Request List
  Option: (255) End

```

下面分析在这个报文中值得讨论的选项: 1. Option(50) Requested IP Address 指示客户端要使用的 IP 地址。2. Option(54) DHCP Server Identifier 指示 DHCP 服务器的信息。由于 Request 报文仍然是广播的, 需要用这个字段来确定在向哪个服务器发出请求, 而其他 DHCP 服务器收到这个广播包后也能知道自己的 Offer 被拒绝了。

最后服务器回应 DHCP ACK 报文, 表示确认成功使用该 IP 地址。

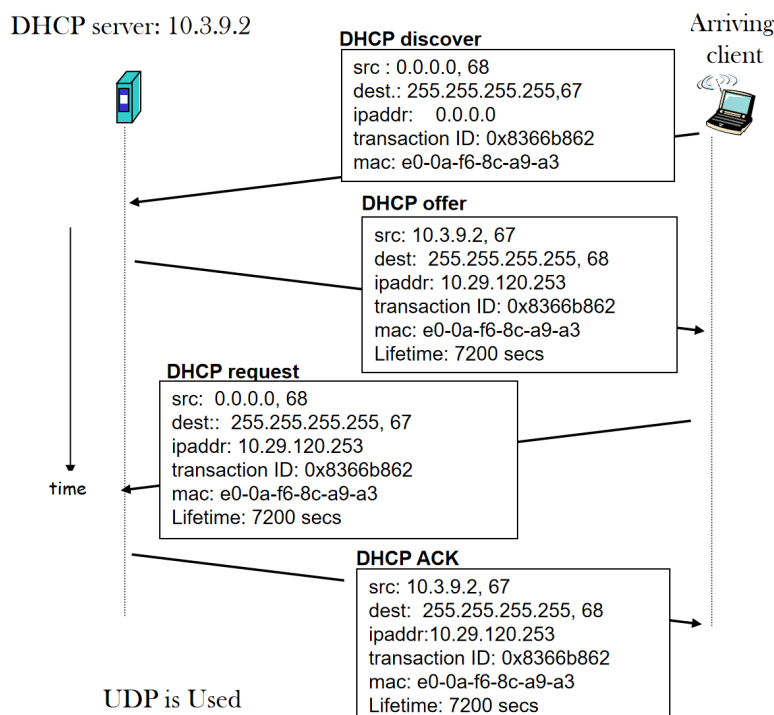
```

Internet Protocol Version 4, Src: 10.3.9.2, Dst: 10.29.120.253
User Datagram Protocol, Src Port: 67, Dst Port: 68
Dynamic Host Configuration Protocol (ACK)
  Message type: Boot Reply (2)
  Hardware type: Ethernet (0x01)
  Hardware address length: 6
  Hops: 1
  Transaction ID: 0x8366b862
  Seconds elapsed: 0
  Bootp flags: 0x0000 (Unicast)
  Client IP address: 0.0.0.0
  Your (client) IP address: 10.29.120.253
  Next server IP address: 0.0.0.0
  Relay agent IP address: 10.29.0.1
  Client MAC address: LiteonTechno_8c:a9:a3 (e0:0a:f6:8c:a9:a3)
  Client hardware address padding: 00000000000000000000
  Server host name not given
  Boot file name not given
  Magic cookie: DHCP
  Option: (53) DHCP Message Type (ACK)
  Option: (54) DHCP Server Identifier (10.3.9.2)
  Option: (51) IP Address Lease Time
  Option: (1) Subnet Mask (255.255.0.0)
  Option: (3) Router
  Option: (6) Domain Name Server
  Option: (255) End
  Padding: 0000000000000000000000000000000000000000

```

对应消息码为 Boot Reply 表示服务器发送给客户端，与 Offer 报文类似的是，也是通过 Router(10.29.0.1) 网关作为中继服务器转发报文，其余字段已在前述分析中详细讲解过

2.3.4 消息序列图



当 DHCP Client 不再需要分配的 IP 地址时，可以通过 DHCP Release 消息通知 DHCP Server 释放该 IP 地址。DHCP Server 收到 DHCP Release 消息后，将该 IP 地址标记为可用状态，以便重新分配给其他 DHCP Client 使用。

2.3.5 进一步分析

根据抓包数据，可以看到 DHCP Server 的 IP 地址为 10.3.9.2，而客户端的 IP 地址为 10.129.118.194，这两个 IP 地址不在同一个子网中，因此可以推断出 DHCP Server 不在客户端

所在的子网内, 需要通过 DHCP Relay 转发 DHCP 请求和响应。

此外, 抓包数据中还可以看到 Relay agent IP address 字段为 10.29.0.1, 这个 IP 地址可以被解释为 DHCP Relay 的 IP 地址。因此可以确认这里的 DHCP 服务是由路由器充当 DHCP Relay 转发的。

为什么有时候 ip 是 169.254.xx.xx?

许多客户端操作系统使用称为自动专用 IP 寻址的东西。即使没有 DHCP 服务器, 此过程也会分配 IP 地址。如果未应答消息, 客户端会选择一个随机的 16 位数字并在其前面加上 169.254.xx 它会执行无偿 ARP 并将该地址分配给自己。

如果本机 ip 是 169.254.xx 地址, 则表示无法访问 DHCP 服务器。电脑将无法访问网络, 因为没有路由器连接到或来自该电脑。通过找出电脑无法看到 DHCP 服务器的原因来解决此问题。

2.4 ARP 协议分析

2.4.1 ARP 各字段功能

ARP (Address Resolution Protocol) 协议是一种用于将 IP 地址转换为 MAC 地址的协议, 它使用请求和响应消息来确定网络上的主机的 MAC 地址。下面是 ARP 消息中各个字段的功能

字段	功能
Hardware type	指定网络上使用的硬件类型 (如以太网)
Protocol type	指定网络层协议类型 (如 IPv4)
Hardware size	指定硬件地址的长度 (通常为 6 个字节, 即 MAC 地址的长度)
Protocol size	指定协议地址的长度 (通常为 4 个字节, 即 IPv4 地址的长度)
Opcode	指定 ARP 消息的类型, 如 ARP 请求或 ARP 响应
Sender MAC address	指定发送 ARP 消息的主机的 MAC 地址
Sender IP address	指定发送 ARP 消息的主机的 IP 地址
Target MAC address	指定 ARP 消息中所带的目标 MAC 地址, 只有在 ARP 响应消息中才有数据
Target IP address	指定 ARP 消息中所带的目标 IP 地址

图 9: ARP 字段详细信息

ARP 协议通过使用这些字段来确定网络上的主机的 MAC 地址, 以便进行通信。当一个主机需要发送数据到另一个主机时, 它可以使用 ARP 协议来查询目标主机的 MAC 地址, 然后将数据包发送到目标主机的 MAC 地址。

2.4.2 ARP 通信过程

在一开始的 IP 协议分析中, 当时对同一无线网中的另一台设备 (我的平板) 进行了 ICMP 通信, 此设备此前不在本机的 ARP 表中

设置过滤器为 icmp or arp, Wireshark 抓包得到如下结果

No.	Time	Source	Destination	Protocol	Details
35	10.767612	LiteonTechno...	Broadcast	ARP	42 Who has 10.129.210.184? Tell 10.129.118.194
37	11.196639	ba:ff:54:fa:1...	LiteonTechno...	ARP	42 10.129.210.184 is at ba:ff:54:fa:1c:78
38	11.196681	10.129.118.194	10.129.210.184	ICMP	74 Echo (ping) request id=0x0001, seq=11/2816, ttl=128 (reply in 39)
39	11.283297	10.129.210.184	10.129.118.194	ICMP	74 Echo (ping) reply id=0x0001, seq=11/2816, ttl=64 (request in 38)
41	11.781952	10.129.118.194	10.129.210.184	ICMP	74 Echo (ping) request id=0x0001, seq=12/3072, ttl=128 (reply in 42)
42	12.037898	10.129.210.184	10.129.118.194	ICMP	74 Echo (ping) reply id=0x0001, seq=12/3072, ttl=64 (request in 41)
47	12.792343	10.129.118.194	10.129.210.184	ICMP	74 Echo (ping) request id=0x0001, seq=13/3328, ttl=128 (reply in 48)
48	13.169820	10.129.210.184	10.129.118.194	ICMP	74 Echo (ping) reply id=0x0001, seq=13/3328, ttl=64 (request in 47)
52	13.801186	10.129.118.194	10.129.210.184	ICMP	74 Echo (ping) request id=0x0001, seq=14/3584, ttl=128 (reply in 53)
53	13.809117	10.129.210.184	10.129.118.194	ICMP	74 Echo (ping) reply id=0x0001, seq=14/3584, ttl=64 (request in 52)

首先, 本设备的 ARP 表中没有 10.129.210.184 的记录, 因此发出一个 ARP 请求报文。ARP 请求报文为广播报文, 内容包括发送方的 IP 地址、MAC 地址, 以及目标的 IP 地址、MAC 地址 (请求时目标的 MAC 地址还不知道, 因此用全 0 占位)。

```
Frame 35: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface \Device\NPF_{FB}
Ethernet II, Src: LiteonTechno_8c:a9:a3 (e0:0a:f6:8c:a9:a3), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
  Destination: Broadcast (ff:ff:ff:ff:ff:ff)
  Source: LiteonTechno_8c:a9:a3 (e0:0a:f6:8c:a9:a3)
  Type: ARP (0x0806)
  Address Resolution Protocol (request)
    Hardware type: Ethernet (1)
    Protocol type: IPv4 (0x0800)
    Hardware size: 6
    Protocol size: 4
    Opcode: request (1)
    Sender MAC address: LiteonTechno_8c:a9:a3 (e0:0a:f6:8c:a9:a3)
    Sender IP address: 10.129.118.194
    Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00)
    Target IP address: 10.129.210.184
```

图 10: ARP 请求

目标设备收到 ARP 请求后返回一个单播的 ARP 回应。内容同样包括发送方的 IP 地址、MAC 地址, 以及目标的 IP 地址、MAC 地址。本设备收到 ARP 回应后就获知了目标设备的 MAC 地址, 并写入本地的 ARP 表。

```
Ethernet II, Src: ba:ff:54:fa:1c:78 (ba:ff:54:fa:1c:78), Dst: Litec
  Destination: LiteonTechno_8c:a9:a3 (e0:0a:f6:8c:a9:a3)
  Source: ba:ff:54:fa:1c:78 (ba:ff:54:fa:1c:78)
  Type: ARP (0x0806)
  Address Resolution Protocol (reply)
    Hardware type: Ethernet (1)
    Protocol type: IPv4 (0x0800)
    Hardware size: 6
    Protocol size: 4
    Opcode: reply (2)
    Sender MAC address: ba:ff:54:fa:1c:78 (ba:ff:54:fa:1c:78)
    Sender IP address: 10.129.210.184
    Target MAC address: LiteonTechno_8c:a9:a3 (e0:0a:f6:8c:a9:a3)
    Target IP address: 10.129.118.194
```

图 11: ARP 响应

网络中时不时会出现 ARP Announcement 消息

ARP Announcement 消息是由源 MAC 地址为主机广播到本地网络的。它的目的是通知其他主机, 它的 IP 地址为 169.254.46.40, 并将其与自己的 MAC 地址绑定。这个消息中的 Sender MAC 地址和 Sender IP 地址字段指定了消息发送者的 MAC 和 IP 地址, 而 Target MAC 地址和 Target IP 地址字段则设置为 00:00:00:00:00:00 和对应本机, 分别用于指定消息中所提到的 IP 地址的 MAC 地址和广播地址。

需要注意的是, 该消息中的 Opcode 字段设置为请求 (request), 表明这是一个 ARP 请求消息, 它在本地网络上向所有主机广播, 以便获取目标 IP 地址的 MAC 地址。同时, 该消息的 Is gratuitous 和 Is announcement 字段都被设置为 True, 表明这是一个无偿和广播的 ARP Announcement 消息。

2.5 TCP 协议分析

2.5.1 TCP 首部字段功能

总结为下表

字段 (字节数)	解释
源端口 (2)	源端口号, 用于标识发送应用程序的进程。
目的端口(2)	目的端口号, 用于标识接收应用程序的进程。
序列号(4)	用于数据重组和排序, 确保数据按正确顺序到达。
接收序列号(4)	如果ACK标志设置, 此字段有效。表示接收方期望从发送方接收的下一个字节的序号。
首部长度(4bits)	表示TCP头部的长度, 以32位字（4字节）为单位。用于指示数据在数据段中的起始位置。
保留位(3bits) 标志位 (9bits)	• NS (Nonce Sum) : 增加TCP抗攻击能力。 • CWR (Congestion Window Reduced) : 拥塞窗口减少。 • ECE (ECN Echo) : ECN回显。 • URG (Urgent) : 紧急指针字段有效。 • ACK (Acknowledgment) : 确认字段有效。 • PSH (Push) : 推送数据到接收应用层。 • RST (Reset) : 复位连接。 • SYN (Synchronize) : 同步序列号, 用于建立连接。 • FIN (Finish) : 发送方完成发送数据, 用于关闭连接。
窗口大小(2)	用于流量控制, 表示接收方可以接收的字节数（从确认号开始计算）。
校验和 (2)	用于检验TCP头部和数据的完整性。
紧急指针(2)	如果URG标志设置, 此字段有效。指示紧急数据的结束位置（序列号的相对偏移量）。
选项(可变)	含 MSS、WScale、SACK 等选项

图 12: 首部字段详细解释

2.5.2 捕获 TCP 数据包

设置 Wireshark 过滤器为 ip.addr == 110.242.68.3 or ip.addr == 110.242.68.4 and tcp.port == 443 , 捕获 HTTPS 协议数据报文

用浏览器打开 www.baidu.com，在页面完全显示后，关闭浏览器, 即可捕获对应 TCP 数据报文

No.	Time	Source	Destination	Protocol	Length	Info
446	5.428793	10.129.118.194	110.242.68.3	TCP	66	55447 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
465	5.440014	110.242.68.3	10.129.118.194	TCP	66	443 → 55447 [SYN, ACK] Seq=0 Ack=1 Min=8192 Len=0 MSS=1386 WS=32 SACK_PERM
466	5.440200	10.129.118.194	110.242.68.3	TCP	54	55447 → 443 [ACK] Seq=1 Ack=1 Win=131584 Len=0
475	5.460415	10.129.118.194	110.242.68.3	TCP	138	55447 → 443 [ACK] Seq=1 Ack=1 Win=131584 Len=1386 [TCP segment of a reassembled PDU]
468	5.440415	10.129.118.194	110.242.68.3	TLSh.	423	Client Hello (SN=www.baidu.com)
469	5.452097	110.242.68.3	10.129.118.194	TCP	60	443 → 55447 [ACK] Seq=1 Ack=1756 Win=32640 Len=0
470	5.455530	110.242.68.3	10.129.118.194	TLSh.	14	Server Hello
471	5.461873	110.242.68.3	10.129.118.194	TCP	14	443 → 55447 [ACK] Seq=1361 Ack=1756 Win=32640 Len=1360 [TCP segment of a reassembled PDU]
472	5.461873	110.242.68.3	10.129.118.194	TCP	14	443 → 55447 [ACK] Seq=2721 Ack=1756 Win=32640 Len=1360 [TCP segment of a reassembled PDU]
473	5.461873	110.242.68.3	10.129.118.194	TLSh.	11	Certificate, Server Key Exchange, Server Hello Done
474	5.462168	10.129.118.194	110.242.68.3	TCP	54	55447 → 443 [ACK] Seq=1756 Ack=5280 Win=131584 Len=0
475	5.464854	10.129.118.194	110.242.68.3	TLSh.	180	Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
476	5.465016	10.129.118.194	110.242.68.3	TLSh.	13	Application Data
477	5.465076	110.242.68.3	10.129.118.194	TCP	11	[TCP Spurious Retransmission] 443 → 55447 [PSH, ACK] Seq=4881 Ack=1756 Win=32640 Len=1119
480	5.475112	10.129.118.194	110.242.68.3	TCP	60	443 → 55447 [ACK] Seq=5280 Ack=1852 Win=32640 Len=0
481	5.474643	110.242.68.3	10.129.118.194	TCP	60	443 → 55447 [ACK] Seq=5280 Ack=1852 Win=32640 Len=0
482	5.477288	110.242.68.3	10.129.118.194	TLSh.	280	New Session Ticket, Change Cipher Spec, Encrypted Handshake Message
483	5.477288	110.242.68.3	10.129.118.194	TCP	60	443 → 55447 [ACK] Seq=5426 Ack=3148 Win=35456 Len=0

图 13: TCP 抓包

2.5.3 连接建立

TCP 协议中的标志位和序号的作用如下：

标志位：

SYN (Synchronize)：用于建立连接，表示请求建立一条连接。

ACK (Acknowledgment)：确认标志位，表示已经收到对方发送的数据。

FIN (Finish)：用于释放连接，表示数据传输完成，请求释放连接。

RST (Reset)：表示重置连接，一般用于异常情况，如连接超时或出现错误。

序号：

序列号：表示发送方发送的数据的第一个字节的序号。

确认号: 表示接收方期望收到的下一个字节的序号。

序号 446, 465, 466: 客户端向服务器发送 SYN 请求建立连接, 服务器返回 SYN+ACK 应答, 客户端返回 ACK 完成连接建立。

其中序号 466 作为客户端对服务器的确认帧, 后面发送的序号 467 的数据帧也可以实现向服务器确认连接建立的功能的帧

在抓包软件中, 所有序号有相对序号与绝对序号两种表示方式

接下来将均用相对序号表示数据包传输过程

第一次握手的序列号为 0, ACK 位设置为 0, 协商的流量窗口大小为 64240

选项中 MSS 设置为 1460 字节, 通过设置 MSS, 可以避免 IP 分片, 提高传输效率和稳定性; 启用了选择性确认功能。SACK 选项允许接收方在发生数据包丢失时, 只重传丢失的段, 而不是从丢失段开始的所有数据。这大大提高了数据传输效率, 尤其是在高丢包率的网络中。

TCP 窗口缩放选项用于支持大于 65535 字节的窗口大小, 这是 TCP 窗口大小的最大值。它通过一个缩放因子 (左移位数) 扩展窗口大小, 允许接收窗口大小达到更高的值。选项中设置了 Window scale = 8, 即可扩展至原窗口的 256 倍

```
▸ Frame 446: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface 0
▸ Ethernet II, Src: LiteonTechno_8c:a9:a3 (e0:0a:f6:8c:a9:a3), Dst: A
▸ Internet Protocol Version 4, Src: 10.129.118.194, Dst: 110.242.68.3
▸ Transmission Control Protocol, Src Port: 55447, Dst Port: 443, Seq:
  Source Port: 55447
  Destination Port: 443
  [Stream index: 11]
  ▸ [Conversation completeness: Complete, WITH_DATA (63)]
  [TCP Segment Len: 0]
  Sequence Number: 0 (relative sequence number)
  Sequence Number (raw): 4079316517
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 0
  Acknowledgment number (raw): 0
  1000 .... = Header Length: 32 bytes (8)
  ▸ Flags: 0x002 (SYN)
  Window: 64240
  [Calculated window size: 64240]
  Checksum: 0x0449 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  ▸ Options: (12 bytes), Maximum segment size, No-Operation (NOP), Wi
  ▸ [Timestamps]
```

图 14: 第一次握手 SYN 请求

第二次握手时服务器协商窗口大小为 8192, 序列号为 0, 对客户端的确认号为 1, 选项中设置 MSS 为 1386 字节, WS 缩放因子为 5, 允许 SACK 机制

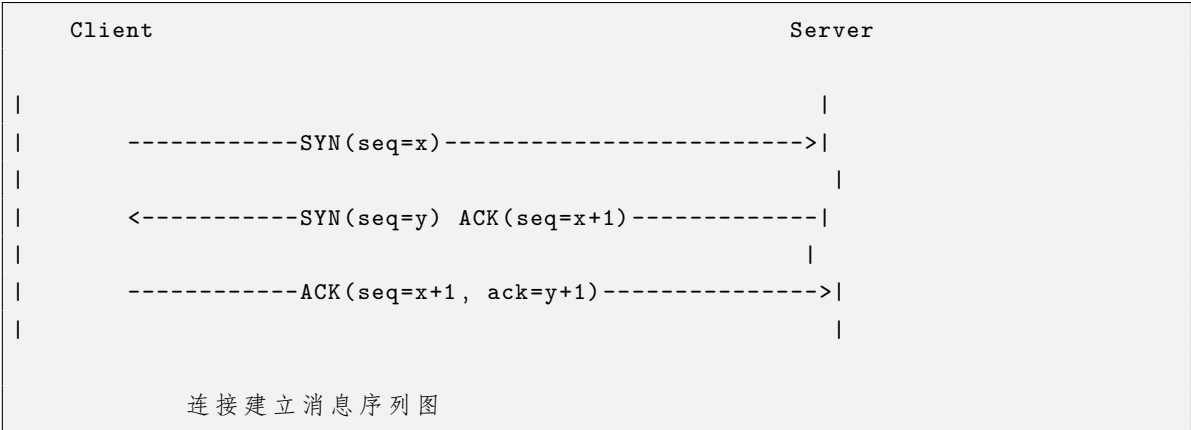
第三次握手 ACK 包序列号为 1, 确认号为 1, 最终协商窗口大小为 514, WS 为 256

```

Frame 465: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface \D
Ethernet II, Src: ArubaHewlett_6c:0c:00 (10:4f:58:6c:0c:00), Dst: LiteonTechno_8c:a9
Internet Protocol Version 4, Src: 110.242.68.3, Dst: 10.129.118.194
Transmission Control Protocol, Src Port: 443, Dst Port: 55447, Seq: 0, Ack: 1, Len:
  Source Port: 443
  Destination Port: 55447
  [Stream index: 11]
  [Conversation completeness: Complete, WITH_DATA (63)]
  [TCP Segment Len: 0]
  Sequence Number: 0 (relative sequence number)
  Sequence Number (raw): 2471907421
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 1 (relative ack number)
  Acknowledgment number (raw): 4079316518
  1000 .... = Header Length: 32 bytes (8)
  Flags: 0x012 (SYN, ACK)
  Window: 8192
  [Calculated window size: 8192]
  Checksum: 0xfbc1 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  Options: (12 bytes), Maximum segment size, No-Operation (NOP), Window scale, No-Op
  [Timestamps]
  [SEQ/ACK analysis]
```

图 15: 第二次握手 SYN-ACK 应答

连接建立过程中的消息序列图如下所示



由图，下面是连接建立过程的描述：

客户端向服务器发送一个 SYN 报文，其中序列号为 x，SYN 标志位为 1，表示客户端请求建立连接。

服务器收到 SYN 报文后，向客户端发送一个 SYN+ACK 报文，其中序列号为 y，确认号为 x+1，SYN 和 ACK 标志位均为 1，表示服务器确认收到了客户端的请求，并请求客户端确认连接。

客户端收到服务器发送的 SYN+ACK 报文后，向服务器发送一个 ACK 报文，其中序列号为 x+1，确认号为 y+1，ACK 标志位为 1，表示客户端确认连接已建立。

在连接建立的过程中，SYN 报文用于请求建立连接，SYN+ACK 报文用于确认收到连接请求并请求确认，ACK 报文用于确认连接建立。序列号用于标识数据流中的每个报文段，确认号用于确认对方已经收到了自己发送的报文段。在 SYN 和 SYN+ACK 报文中，序列号用于初始化数据流的起始序号，确认号则用于确认收到的报文序列号。在 ACK 报文中，序列号用于确认收到的报文序列号，确认号则用于确认对方已经收到了自己发送的报文序列号。

```

> Frame 466: 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on inter
> Ethernet II, Src: LiteonTechno_8c:a9:a3 (e0:0a:f6:8c:a9:a3), Dst: ArubaHewle
> Internet Protocol Version 4, Src: 10.129.118.194, Dst: 110.242.68.3
> Transmission Control Protocol, Src Port: 55447, Dst Port: 443, Seq: 1, Ack: 3
  Source Port: 55447
  Destination Port: 443
  [Stream index: 11]
  [Conversation completeness: Complete, WITH_DATA (63)]
  [TCP Segment Len: 0]
  Sequence Number: 1 (relative sequence number)
  Sequence Number (raw): 4079316518
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 1 (relative ack number)
  Acknowledgment number (raw): 2471907422
  0101 .... = Header Length: 20 bytes (5)
  Flags: 0x010 (ACK)
  Window: 514
  [Calculated window size: 131584]
  [Window size scaling factor: 256]
  Checksum: 0x5a46 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  [Timestamps]
  [SEQ/ACK analysis]

```

图 16: 第三次握手 ACK 应答

2.5.4 连接断开

序号 903-907 数据包展示了客户端和服务端分别发送 FIN 请求, 对方返回 FIN+ACK 应答, 最后发送 ACK 完成连接释放。

```

> Frame 466: 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on inter
> Ethernet II, Src: LiteonTechno_8c:a9:a3 (e0:0a:f6:8c:a9:a3), Dst: ArubaHewle
> Internet Protocol Version 4, Src: 10.129.118.194, Dst: 110.242.68.3
> Transmission Control Protocol, Src Port: 55447, Dst Port: 443, Seq: 1, Ack: 3
  Source Port: 55447
  Destination Port: 443
  [Stream index: 11]
  [Conversation completeness: Complete, WITH_DATA (63)]
  [TCP Segment Len: 0]
  Sequence Number: 1 (relative sequence number)
  Sequence Number (raw): 4079316518
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 1 (relative ack number)
  Acknowledgment number (raw): 2471907422
  0101 .... = Header Length: 20 bytes (5)
  Flags: 0x010 (ACK)
  Window: 514
  [Calculated window size: 131584]
  [Window size scaling factor: 256]
  Checksum: 0x5a46 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  [Timestamps]
  [SEQ/ACK analysis]

```

图 17: TCP 断开连接

接下来进行详细分析

第一次挥手客户端发送的 FIN-ACK 数据包序号为 3148, 确认号为 114099

第二次挥手服务器发送的 ACK 数据包序号为 114099, 确认号为 3149, 协商窗口大小为 1108

第三次挥手服务端发送的 FIN-ACK 数据包序号为 114130, 确认号为 3149, 协商窗口大小为 1108

第四次挥手服务端发送的 FIN-ACK 数据包序号为 3149, 确认号为 114130, 协商窗口大小为 0

在连接释放的过程中, 序列号和确认号的作用与连接建立过程中相同, 用于确认对方是否确

```
Internet Protocol Version 4, Src: 20712, Dst: 110712, Seq: 1107210075
Transmission Control Protocol, Src Port: 55447, Dst Port: 443, Seq: 3148, Win: 514
  Source Port: 55447
  Destination Port: 443
  [Stream index: 11]
  [Conversation completeness: Complete, WITH_DATA (63)]
  [TCP Segment Len: 0]
  Sequence Number: 3148 (relative sequence number)
  Sequence Number (raw): 4079319665
  [Next Sequence Number: 3149 (relative sequence number)]
  Acknowledgment Number: 114099 (relative ack number)
  Acknowledgment number (raw): 2472021520
  0101 .... = Header Length: 20 bytes (5)
  Flags: 0x011 (FIN, ACK)
  Window: 514
  [Calculated window size: 131584]
  [Window size scaling factor: 256]
  Checksum: 0x9046 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  [Timestamps]
```

图 18: 第一次挥手

```
Transmission Control Protocol, Src Port: 443, Dst Port: 55447, Seq: 114099, Win: 1108
  Source Port: 443
  Destination Port: 55447
  [Stream index: 11]
  [Conversation completeness: Complete, WITH_DATA (63)]
  [TCP Segment Len: 0]
  Sequence Number: 114099 (relative sequence number)
  Sequence Number (raw): 2472021520
  [Next Sequence Number: 114099 (relative sequence number)]
  Acknowledgment Number: 3149 (relative ack number)
  Acknowledgment number (raw): 4079319666
  0101 .... = Header Length: 20 bytes (5)
  Flags: 0x010 (ACK)
  Window: 1108
  [Calculated window size: 35456]
  [Window size scaling factor: 32]
  Checksum: 0x8df4 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  [Timestamps]
  [SEQ/ACK analysis]
```

图 19: 第二次挥手

认了自己发送的关闭连接请求，从而完成连接的释放。

连接释放部分的消息序列图如下所示：



1. 第一次挥手

序号 seq 字段的值设置为 u，它等于 TCP 客户进程之前已经传送过的数据的最后一个字节的序号加 1。

```

Transmission Control Protocol, Src Port: 443, Dst Port: 55447, Seq: 114130,
Source Port: 443
Destination Port: 55447
[Stream index: 11]
  [Conversation completeness: Complete, WITH_DATA (63)]
  [TCP Segment Len: 0]
  Sequence Number: 114130 (relative sequence number)
  Sequence Number (raw): 2472021551
  [Next Sequence Number: 114131 (relative sequence number)]
  Acknowledgment Number: 3149 (relative ack number)
  Acknowledgment number (raw): 4079319666
  0101 ... = Header Length: 20 bytes (5)
  [Flags: 0x011 (FIN, ACK)]
  Window: 1108
  [Calculated window size: 35456]
  [Window size scaling factor: 32]
  Checksum: 0x8dd4 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  [Timestamps]

```

图 20: 第三次挥手

```

Internet Protocol Version 4, Src: 10.129.110.124, Dst: 110.74.21.60
Transmission Control Protocol, Src Port: 55447, Dst Port: 443, Seq: 3149,
Source Port: 55447
Destination Port: 443
[Stream index: 11]
  [Conversation completeness: Complete, WITH_DATA (63)]
  [TCP Segment Len: 0]
  Sequence Number: 3149 (relative sequence number)
  Sequence Number (raw): 4079319666
  [Next Sequence Number: 3149 (relative sequence number)]
  Acknowledgment Number: 114130 (relative ack number)
  Acknowledgment number (raw): 2472021551
  0101 ... = Header Length: 20 bytes (5)
  [Flags: 0x014 (RST, ACK)]
  Window: 0
  [Calculated window size: 0]
  [Window size scaling factor: 256]
  Checksum: 0x9225 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  [Timestamps]
  [SEQ/ACK analysis]

```

图 21: 第四次挥手

TCP 规定终止标志位 FIN 等于 1 的 TCP 报文段即使不携带数据，也要消耗掉一个序号。确认号 ack 字段的值设置为 v，它等于 TCP 客户进程之前已收到的数据的最后一个字节的序号加 1。

2. 第二次挥手

序号 seq 字段的值设置为 v，它等于 TCP 服务器进程之前已传送过的数据的最后一个字节的序号加 1。这也与之前收到的 TCP 连接释放报文段中的确认号 v 匹配。

确认号 ack 字段的值设置为 u+1，这是对 TCP 连接释放报文段的确认。

经过两次挥手后，从 TCP 客户进程到 TCP 服务器进程这个方向的连接就释放了，但 TCP 服务器进程如果还有数据要发送，TCP 客户进程仍要接收，也就是从 TCP 服务器进程到 TCP 客户进程这个方向的连接并未关闭。

3. 第三次挥手

TCP 连接释放报文段首部中的终止标志位 FIN 和确认标志位 ACK 的值都被设置为 1。表明这是一个 TCP 连接释放报文段，同时也对之前收到的 TCP 报文段进行确认。

序号 seq 字段的值假定被设置为 w，这是因为在半关闭状态下 TCP 服务器进程可能又发送了一些数据。

确认号 ack 字段的值被设置为 u+1, 这是对之前收到的 TCP 连接释放报文段的重复确认。
TCP 服务器进入 LAST-ACK 状态

4. 第四次挥手

序号 seq 字段的值设置为 u+1, 这是因为 TCP 客户进程之前发送的 TCP 连接释放报文段虽然不携带数据, 但要消耗掉一个序号。

确认号 ack 字段的值设置为 w+1, 这是对所收到的 TCP 连接释放报文段的确认。

TCP 客户端进入 TIME-WAIT 状态, 等待 2MLS, MSL 是最长报文段寿命 (Maximum Segment Lifetime) 的英文缩写词, [RFC793] 建议为 2 分钟。也就是说, TCP 客户进程进入时间等待 (TIME-WAIT) 状态后, 还要经过 4 分钟才能进入关闭 (CLOSED) 状态。

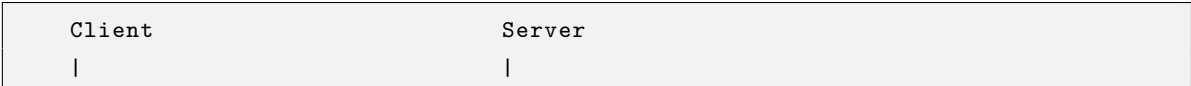
2.5.5 数据传输过程

抓包过程中的数据传输如下

487	5.482275	110.242.68.3	10.129.118.194	TCP	14.443 → 55447 [ACK] Seq=6634 Ack=3148 Win=35456 Len=1360 [TCP segment of a reassembled PDU]
488	5.482275	110.242.68.3	10.129.118.194	TLSv..	18. Application Data
489	5.482275	110.242.68.3	10.129.118.194	TLSv..	642. Application Data
490	5.482368	10.129.118.194	110.242.68.3	TCP	54 55447 → 443 [ACK] Seq=3148 Ack=9089 Win=131584 Len=0
491	5.482433	110.242.68.3	10.129.118.194	TCP	14.443 → 55447 [ACK] Seq=9089 Ack=3148 Win=35456 Len=1360 [TCP segment of a reassembled PDU]
492	5.482433	110.242.68.3	10.129.118.194	TCP	14.443 → 55447 [ACK] Seq=10969 Ack=3148 Win=35456 Len=1360 [TCP segment of a reassembled PDU]
493	5.482433	110.242.68.3	10.129.118.194	TCP	14.443 → 55447 [ACK] Seq=12329 Ack=3148 Win=35456 Len=1360 [TCP segment of a reassembled PDU]
494	5.482433	110.242.68.3	10.129.118.194	TLSv..	99. Application Data
495	5.482487	10.129.118.194	110.242.68.3	TCP	54 55447 → 443 [ACK] Seq=3148 Ack=13734 Win=131584 Len=0
496	5.482825	110.242.68.3	10.129.118.194	TCP	14.443 → 55447 [ACK] Seq=13734 Ack=3148 Win=35456 Len=1360 [TCP segment of a reassembled PDU]
497	5.482825	110.242.68.3	10.129.118.194	TCP	14.443 → 55447 [ACK] Seq=15904 Ack=3148 Win=35456 Len=1360 [TCP segment of a reassembled PDU]
498	5.482825	110.242.68.3	10.129.118.194	TCP	14.443 → 55447 [ACK] Seq=16454 Ack=3148 Win=35456 Len=1360 [TCP segment of a reassembled PDU]
499	5.482825	110.242.68.3	10.129.118.194	TCP	14.443 → 55447 [ACK] Seq=17814 Ack=3148 Win=35456 Len=1360 [TCP segment of a reassembled PDU]
500	5.482825	110.242.68.3	10.129.118.194	TLSv..	138. Application Data
501	5.482825	110.242.68.3	10.129.118.194	TCP	14.443 → 55447 [ACK] Seq=19658 Ack=3148 Win=35456 Len=1360 [TCP segment of a reassembled PDU]

图 22: 数据传输

- 字段作用分析:
- 1. 发送序号 (Seq):
作用: 用于标识数据段在整个数据流中的位置。每个字节的数据都有一个唯一的序号。
示例: 如果发送的第一个字节序号是 1000, 长度为 100 字节, 那么下一个段的序号是 1100。
 - 2. 应答序号 (Ack):
作用: 表示接收方期望接收的下一个字节序号, 即确认已经接收到的数据的最后一个字节序号加 1。
示例: 如果接收方收到的最后一个字节的序号是 1100, 则应答序号为 1101。
 - 3. 应答标志位 (ACK):
作用: 用于标识该段是一个确认段。ACK 标志位为 1 表示该段包含有效的确认号。
示例: 当接收方收到数据并发送 ACK 段时, 该段的 ACK 标志位被设置为 1。
 - 4. 窗口大小 (Window Size):
作用: 表示接收方当前可接受的缓冲区大小, 用于流量控制。
示例: 窗口大小为 4096 字节, 表示接收方可以接受最多 4096 字节的数据而不会发生溢出。
 - 5. 数据长度:
作用: 表示 TCP 段中实际承载的数据的长度。
示例: 如果一个 TCP 段包含 500 字节的数据, 那么数据长度为 500。
 - 6. 最大分段大小 (MSS):
作用: 用于告知对方发送的最大 TCP 段大小, 帮助避免 IP 分片。
示例: MSS 为 1460 字节, 表示每个 TCP 段的最大数据部分为 1460 字节。
- 对应数据传输序列图:



```
| --- [Seq=1, Len=1000] ---> | (Data Segment 1)
|
| --- [Seq=1001, Len=1000] --> | (Data Segment 2)
|
| <--- [ACK=1001] ----- | (Ack for Segment 1)
| <--- [Seq=2000, Len=1000] -- | (Server sends data, causes error)
|
| --- [ACK=2001] -----> | (Client acknowledges erroneous data)
|
| --- [Seq=1001, Len=1000] --> | (Retransmit Data Segment 2)
|
| <--- [ACK=2001] ----- | (Ack for Retransmitted Segment 2)
|
|
```

3 实验结论和心得

本次实验难度不大, 整体进展顺利。

本次实验主要通过实际捕获和分析网络数据, 加深对 IP、TCP、UDP、ICMP、ARP 和 DHCP 等协议的理解。通过分析各个协议的数据包格式, 理解了各个字段的含义和作用, 对协议的工作原理有了更加直观的认识。

在实验过程中, 理解数据包头格式和字段信息存在一定难度。通过多次检查教材和资料, 分析模拟, 逐渐理清了各个字段的含义。在分析 DHCP、ARP 消息序列时, 需要多次观察捕获的消息, 结合资料理解协议的工作过程。此外, 分析 TCP 连接建立、释放和数据传输过程时, 对于序号、窗口管理等概念的理解不够深刻, 通过多次调试和修改, 才能总结出较为合理的消息序列图。

总的来说, 本次实验给我带来很多收获。通过实际操作, 一是巩固和扩展了对协议工作原理的理解, 二是学习了实用的网络分析工具。在后续的网络学习和工作中, Wireshark 软件将作为研究网络问题和故障排除的必要工具。